Filesystem-Gated Multi-Agent Workflow for Scientific Software Development

AI Supervisor/Worker Workflow

May 31, 2026

Abstract

This document describes a reusable AI-assisted coding workflow for scientific software projects. A milestone-gated supervisor decomposes large goals into small jobs, one or more implementation workers execute those jobs, and optional read-only reviewers inspect each worker attempt before acceptance. The default deployment uses Codex CLI for the supervisor and Cursor Agent for workers and reviewers, but those command-line wrappers are configurable per role. All coordination happens through durable files in the project repository, which makes state auditable, restartable, and independent of direct agent-to-agent process control.

1 Design Goals

The workflow is designed for scientific codes where correctness, reproducibility, and reviewable history matter more than maximizing autonomous throughput. The main goals are:

- decompose large milestones into small, closed-form worker jobs;
- require tests and scientific assumptions for numerical work;
- preserve reviewable Git history and attempt documentation;
- keep human intervention at milestone gates rather than every job;
- recover from crashes, timeouts, stale locks, and machine restarts;
- make reviewer coverage and blocking decisions machine-checkable;
- collect metrics for later analysis of agent reliability and cost.

2 Repository Layout

Path	Purpose
`AGENTS.md`	Role, scientific coding, Git hygiene, and workflow-evolution rules.
`.ai/supervisor/`	Durable supervisor memory: design prompt, project brief, roadmap, ledger, policies, and human gates.
`.ai/jobs/JNNNN/`	One worker job: task, status, reports, logs, patches, reviewer reports, and feedback.

`.ai/commit_docs/`	Markdown documentation for each worker attempt and its commit range.
`.ai/metrics/`	Structured run metrics and workflow events.
`.worktrees/JNNNN/`	Isolated Git worktree for a worker job branch.
`agent_wrappers/`	Role-neutral wrapper registry with executable, model, and recommended-role metadata.
`gui/`	Local dashboard for launching/stopping loops, selecting wrappers/models, reviewing gates, and inspecting logs.
`skills/`	Project skills plus installed reusable workflow skills visible to Codex and Cursor prompts.
`scripts/`	Dependency-light shell and Python helpers for dispatch, review, metrics, GUI, and recovery.

3 Roles

3.1 Supervisor

The supervisor owns planning, dispatch, review, acceptance, rejection, and ledger maintenance. In the default configuration it is launched through Codex CLI, but the dashboard can also launch it through Cursor Agent or any future compatible wrapper. The supervisor reads the project design and current state, creates one small job at a time, reviews the worker report, tests, diff, commit docs, and reviewer reports, then integrates or rejects the job. Supervisor-owned planning files include `roadmap.md`, `project_brief.md`, and `ledger.md`; workers may suggest changes to those files but should not own them.

3.2 Worker

The worker implements exactly one assigned job in an isolated worktree. In the default configuration it is launched through Cursor Agent, but Codex CLI can also be selected as the worker wrapper. The worker runs the requested validation, keeps changes in meaningful commits where possible, and writes a concise report. Each report includes workflow friction and skill suggestions so repeated procedural failures can be converted into templates, scripts, protocols, or skills.

3.3 Reviewers

Two optional read-only reviewers run after the worker and tests finish. Each reviewer has an independent wrapper and model. Reviewer A focuses on scientific, mathematical, numerical, and algorithmic robustness. Reviewer B focuses on code quality, build behavior, portability, tests, maintainability, and Git hygiene. Reviewers inspect the actual diff rather than relying on the worker report.

4 Agent Wrappers and Model Selection

Agent wrappers decouple workflow roles from a specific command-line product. The registry in `agent_wrappers/` declares wrapper IDs, labels, executables, supported roles, recommended roles, default models per role, and the models that should appear in the dashboard dropdowns. The built-in wrappers are role-neutral:

Wrapper	Recommended roles	Default model behavior
`codex`	Supervisor and workflow chat	Uses Codex model IDs such as `gpt-5.5` with a separate reasoning-effort control.
`cursor-agent`	Worker and reviewers	Uses Cursor model IDs such as `gpt-5.5-high`, `gpt-5.3-codex-high`, and `claude-opus-4-7-thinking-high`.

The same wrappers can be selected for non-recommended roles when desired. For example, the dashboard can launch a Codex CLI worker or a Cursor Agent supervisor. Model dropdowns are populated from the selected wrapper metadata, which avoids mixing Codex model names with Cursor model names unless a wrapper explicitly advertises them.

New wrappers, such as a future Claude Code integration, should be added by creating `agent_wrappers/<wrappe` and either a command template or a built-in runner in `scripts/agent_wrapper.py`. The workflow loops call `scripts/agent_wrapper.py run`; they do not hard-code a vendor command except inside the wrapper runner.

5 Dashboard

The local dashboard is served by `scripts/workflow_gui.py`. It provides:

- launch and stop controls for worker and supervisor loops;
- wrapper and model dropdowns for worker, reviewer A, reviewer B, and supervisor;
- current run status, job state, milestone criteria, project tree, and ledger views;
- bounded real-time log panels for worker, supervisor, reviewer, and GUI output;
- a human milestone-review form with yes/no checklist answers, comments, an approval comment, and a major-structural-change override;
- workflow-aware chat panels for asking questions before submitting human review.

The dashboard stores a project-local port record in `.ai/runtime/workflow_gui_port.json`. If no explicit `--port` is given, a project reuses its stored port; a new project chooses the first available port at or above 8765 and stores it. This lets multiple project dashboards run at the same time without manual port bookkeeping.

When a human gate exists, the worker loop pauses and the supervisor loop exits with `HUMAN_REVIEW_REQUIRED`. This is intentional: the dashboard remains available for the human review, and after the review is submitted the workflow can restart the loops automatically according to the project configuration.

6 Overall Control Flow

```
Human design prompt
        |
        v
Supervisor extracts brief, roadmap, and current milestone
```

```
        |
        v
Create exactly one small worker job in .ai/jobs/JNNNN/
        |
        v
Selected worker wrapper implements in .worktrees/JNNNN
        |
        v
Validation, commit docs, changed_files, diffstat, report
        |
        v
Selected reviewer wrappers inspect the actual diff in parallel
        |
        v
Supervisor reviews worker artifacts plus reviewer artifacts
        |
        +-- reject --> feedback.md --> worker retry
        |
        +-- accept --> integrate branch, update ledger
                         |
                         +-- milestone incomplete --> next small job
                         |
                         +-- milestone complete/blocked --> human review gate
```

7 Job State Machine

Each job has a `status.json` with immutable `base_sha`, human-readable `base_ref`, the job branch, attempt number, and current state. The immutable base commit is the review boundary for all diffs.

```
queued/rejected
      |
      v
running -- deterministic or worker failure --> blocked
      |
      v
reviewing -- reviewer timeout/failure/block --> review_failed or review_timeout
      |
      v
ready_for_review -- supervisor rejects --> rejected
      |
      +-- supervisor accepts --> accepted

Any nonterminal job can become cancelled or superseded by explicit decision.
Terminal jobs are not retried by the worker loop.
```

8 Worker Attempt Protocol

The worker loop performs deterministic steps before and after invoking Cursor:

1. Acquire a loop-level singleton lock and a job lock with owner PID.
2. Create or reuse `.worktrees/JNNNN` on branch `ai/JNNNN`.
3. Resolve and persist `base_sha`; never use a moving ref for review.
4. Run preflight checks: branch, worktree cleanliness, base ancestry, and project-configured submodule requirements.
5. Build a prompt containing only the job task, feedback, visible skills, validation command, and report contract.
6. Invoke the selected worker wrapper with configurable model, timeout, and extra local flags.
7. Commit remaining changes if Cursor did not create commits.
8. Run the job validation command, optionally bounded by `TEST_TIMEOUT`.
9. Record raw and filtered post-test dirty status. Allowed generated files may be declared in `allowed_artifacts.txt`; undeclared dirty files block the attempt.
10. Generate `changed_files`, `diffstat`, `diff.patch`, `worker_handoff.attempt-N.json`, `report.md`, commit docs, consistency checks, metrics, and events.

The raw worker transcript is retained as audit evidence only. The worker loop extracts a structured handoff from the final worker report and uses canonical workflow facts plus that handoff to render `report.md` and commit documentation. Raw transcript files such as `cursor_final.attempt-N.md` and `cursor_stream.attempt-N.jsonl` are not embedded into canonical review artifacts.

9 Reviewer Protocol

Reviewers run in parallel through the selected reviewer wrappers. Each report must include a fenced YAML block:

```
diff_coverage:
  full_diff_reviewed: true
  files_reviewed:
    - path/from/changed_files
  unreviewed_files: []
review_decision:
  recommendation: accept
  blocks_acceptance: false
  blocking_reasons: []
```

The worker loop checks coverage with `check_reviewer_coverage.py` and parses reviewer decisions with `analyze_reviewer_reports.py`. Reviewer timeouts or blocking recommendations prevent the job from becoming ready for supervisor acceptance.

10 Supervisor Acceptance Protocol

The supervisor reviews the canonical artifacts rather than relying on a single report. It should inspect:

- `status.json`, including `base_sha`, attempts, tests, and reviewer fields;
- `worker_handoff.attempt-N.json`, `report.md`, `test.attempt-N.log`, `diffstat`, and selected patch sections;
- `changed_files.attempt-N.txt` and reviewer coverage;
- reviewer A and B reports and reviewer-decision analysis;
- commit documentation and the actual branch diff `base_sha..HEAD`;
- attempt-consistency and post-test-dirty artifacts.

Before integration, `integrate_job.py` verifies state, tests, reviewer completion, post-test cleanliness, attempt consistency, branch existence, and base ancestry. Integration is explicit and should be done from a clean main worktree.

11 Human Gates

Human review is reserved for milestone boundaries, major structural changes, and decisions the supervisor cannot safely make. The dashboard cycles through every checklist item, records yes/no answers and comments, and supports an optional approval comment. Ordinary failed checklist items are routed through `HUMAN_REVIEW_ACTION_REQUESTED.md`; the supervisor classifies them and creates a small revision job, updates planning records, or opens a clarification gate. Major structural requests supersede the checklist outcome and are routed through `STRUCTURAL_CHANGE_REQUESTED.md`. The supervisor must update the milestone plan first, then return to the human with a revised review gate before implementation continues.

12 Failure Handling

Failure mode	**Handling**
Worker timeout	Record timeout, optionally auto-resume if configured, otherwise block.
Worker crash	Auto-relaunch limited attempts for non-hard failures; record event.
Reviewer timeout/crash	Retry reviewers, then set `review_timeout` or `review_failed`.
Reviewer blocks acceptance	Keep job out of `ready_for_review`; supervisor decides reject, waive, or rerun.
Post-test dirty files	Block unless all paths match `allowed_artifacts.txt`.
Stale running job	Recover only if lock owner PID is gone; mark `implemented` if artifacts exist.

Supervisor crash	Create `SUPERVISOR_ACTION_REQUIRED.md`, pause worker dispatch, and avoid a human milestone gate unless a real human decision is needed.
Superseded work	Use terminal state `superseded` so the worker loop does not retry it.

13 Workflow Evolution

Workers report friction and skill suggestions. Reviewers evaluate those suggestions. The supervisor deduplicates them against existing skills, templates, scripts, and protocols, then chooses the smallest durable fix: ledger note, documentation, checklist, template, protocol, script, project skill, or reusable workflow skill. Repeated failure artifacts can also trigger supervisor-originated improvements, even when workers do not recognize the pattern.

14 Metrics and Paper Reproducibility

Worker and reviewer runs write per-agent metrics next to job artifacts, and supervisor runs write metrics under `.ai/metrics/supervisor/`. All records are appended to `.ai/metrics/runs.jsonl`. Workflow events such as failures, reviewer blocks, wrapper crashes, and human interventions are appended to `.ai/metrics/events.jsonl`. These records can support a later empirical study of agent reliability, failure modes, human-intervention frequency, token use, wall-clock time, wrapper choice, model choice, and scientific-code review outcomes.

15 Minimal Commands

```
python3 scripts/workflow_gui.py --host 127.0.0.1 --project-root .

WORKER_AGENT_WRAPPER=cursor-agent \
WORKER_MODEL=gpt-5.5-high \
REVIEWER_A_AGENT_WRAPPER=cursor-agent \
CURSOR_REVIEWER_A_MODEL=claude-opus-4-7-thinking-high \
REVIEWER_B_AGENT_WRAPPER=cursor-agent \
CURSOR_REVIEWER_B_MODEL=gpt-5.3-codex-high \
./scripts/worker_loop.sh

SUPERVISOR_AGENT_WRAPPER=codex \
CODEX_MODEL=gpt-5.5 \
CODEX_REASONING_EFFORT=high \
./scripts/supervisor_loop.sh

python3 scripts/agent_wrapper.py list --json
python3 scripts/summarize_jobs.py
python3 scripts/summarize_agent_metrics.py
```